

# Lab 3: Auditing Your ETL Pipeline · Tonight in 20 Steps

OPIM 5512 · Module 3 · Solo · Colab · Dr. Wanik's backup data first, then your own Google Cloud bucket

**The one-sentence version: before you trust a model, check the data your pipeline is feeding it.** You load your own cars, read the real ads, measure what the regex extractor got right, and fix the worst field with one regular expression.

## Before we start

- [ ] **5 green checks** in your fork's **Actions** tab (Final GCP Guide F26, Steps 0 to 10)
- [ ] Your bucket has `structured/datasets/listings_master.csv` (Force run scraper → extractor → materialize-master)
- [ ] The lab notebook is open in Colab: HuskyCT → In-Class Labs → **Lab 3: Auditing Your ETL Pipeline**

### ⚠ The 3 things that trip everyone up (read this first)

1. **The Colab sign-in pop-up: pick the Gmail you used for Google Cloud**, not necessarily your UConn account. Wrong account = "permission denied" or an empty bucket.
2. **listings\_master.csv not found?** In Cloud Scheduler, **tick the box first**, then Force run the **extractor**, wait for **Success**, then **materialize-master**. One at a time.
3. **Save your work:** the notebook opens from GitHub, so first do **File** → **Save a copy in Drive**. Otherwise your edits disappear when you close the tab.

**Tonight everyone starts on the backup data.** The `USE_BACKUP_DATA` box in Part 2a is already ticked: Dr. Wanik's 29 Hartford cars, no sign-in. When your own pipeline is running, untick it and re-run from 2a.

## The 20 steps, in order

### Part 1 · Get it running (steps 1–5, 30 min; skip to step 6 if your pipeline is not ready, the backup data has you covered)

1. Open the notebook from HuskyCT → **File** → **Save a copy in Drive** (work in the copy).
2. Your fork's **Actions** tab: 5 green checks? If not, work the **Final GCP Guide F26** now. Stuck? Paste the step, the command, and the full error into Claude.
3. **Cloud Scheduler** → tick `craigslist-scraper-hourly` → **Force run** → wait for **Success**.
4. Tick `extractor-per-listing-hourly` → **Force run** → **Success**. Then `materialize-master-hourly` → **Force run** → **Success**.
5. **Cloud Storage** → your bucket → `structured/datasets/listings_master.csv` is there.

---

## Part 2 · Search and see (steps 6–10, 25 min)

6. **2a:** leave the `USE_BACKUP_DATA` box **ticked** (Dr. Wanik's Hartford cars) → run both cells. (Later: *untick it, type your `PROJECT_ID`, re-run from 2a, and pick your Google Cloud Gmail.*)
7. **2b:** run it → click the **folder icon** on the left → `my_cars` → each car has a `.txt` (the raw ad), a `.json` (what the extractor pulled out), and with the backup data a `.html` (the web page) and a `.jpg` (a snapshot of it: double-click to see the ad). Backup data? Open **01** (the extractor copied the seller's `Hyunda` typo) and **03** (make = `Contact`, model = `Information`).
8. **2c:** change `SEARCH_WORD` to a brand ( `toyota`, `honda`, `ford` ) → run → each hit shows its file number and the line it was found in. Why does a Hyundai show up for `toyota`? (File 03 is a keyword stuffer!)
9. **2d:** run it → each car **three ways**: the web page, the saved text, the extracted fields. Start with **01, 03, 05**.
10. Write 2–3 sentences: what does the raw text keep from the web page, and what does it lose?

---

## Part 3 · Validate the extractor against ground truth (steps 11–14, 30 min)

11. **3a:** run it → files 01 to 05 and what the extractor pulled out of each.
12. **Check the ground truth.** With the backup data it is already filled in: open at least **two** `.txt` files and confirm the price, year, make, model and mileage. **Rules:** numbers without \$ or commas · make and model in lowercase · model = first word · the **real** make (a `Hyunda` is a `hyundai`). (Own data: *paste the skeleton and type your own.*)
13. **3b:** run it → the **before** table. Which fields does the extractor get right? Which does it miss?
14. **3c:** run it → what is the old regex grabbing? ( `Contact Information` ? A town?) Then find the **anchor**: a line that is just the **year**, with **make model** on the next line (file 03, lines 39 and 40). The anchor is safe because Craigslist's posting form writes that year line (the seller picks it from a list), so it is always 4 digits, never `'23`. Two quirks to know: a seller's own title can say `'23` or be in ALL CAPS (that is why the title is unreliable), and once in a while the make/model line is a single word, like `Pontiac` with no model (file 08 in the backup data).

---

## Part 4 · Fix it three ways (steps 15–18, 30 min)

15. **Way 1** (given): the 3c anchor turned into code → run it: files 03 and 04 go from `Contact Information` to real cars. Then read *Why isn't Way 1 perfect?* (seller typos, one-word lines, two-word names like `3 series`).
16. **Way 2** (yours): run the **4-step build**. Steps 1 to 3 are given and print what they match in file 03. **Step 4 is yours:** paste `\s+([A-Za-z0-9-]+)` inside the quotes, re-run, and you should see ( `'hyundai'`, `'elantra'` ). Stuck? The full answer is behind the *Still stuck?* click.
17. **Way 3** (stretch): the typo-fixer ( `hyunda` → `hyundai` ) → the `difflib` demo (set `CUTOFF = 0.70` and watch `contact` become a Pontiac) → the **yellow review flags** and the `model_ready` table. Want more makes? Use the Claude prompt in the notebook.
18. **4b + 4c:** the re-score on your 5 cars (original extractor vs way 1 vs **your regex** vs way 3), then **4c: every car, before and after**, with the flagged rows lit up. Fill in the one-sentence summary.

---

## Part 5 · Wrap up (steps 19–20, 5 min)

19. Run the **save** cell (your truth table goes to your bucket for **A07**, where Gemini gets scored on the same 5 cars) → answer the **3 short questions**.
20. **File** → **Download** → **Download .ipynb** → HuskyCT → **Submit Lab 3** (due **Thu Oct 8, 11:59 PM**, both campuses).

---

**Definition of done:** a checked truth table for 5 cars · your step 4 regex piece · the 4b and 4c before-vs-after tables · 3 short answers · submitted. **Leave your pipeline running.** It adds up to 10 new cars every hour, and your midterm uses them.